

denoted as an 'Assignment Edge'. An edge exists from a thread to a resource if the thread has requested for that resource, and such an edge is referred to as a 'Request Edge'.

Consider the resource allocation graph (Figure 1) for our example code, in which we have two threads, T1 and T2; and two resources, Row and Column. One partition, say Partition 1, contains T1 and T2. The other partition, say Partition 2, contains Row and Column resources.

Since 'row_lock' is acquired by Thread 1, an edge exists from the resource 'row' to the node T1. Since 'column_lock' is acquired by Thread 2, an edge exists from the resource 'column' to node T2. Since Thread 1 has requested for the resource 'column', an edge exists from the node T1 to the node 'column'. Similarly, an edge exists from node T2 to node 'row' since Thread 2 has requested for the resource 'column'.

An application is deadlocked if the resource allocation graph of its current state contains a directed cycle in which each 'request edge' is followed by an 'assignment edge'. We can use this fact for deadlock detection. We can write an algorithm to construct the resource allocation graph of an application's current state and look for cycles in it. If a cycle of the kind mentioned above is found, we can declare that the application is deadlocked. I leave it to the reader to write the code for this problem.

There are three strategies programmers use for handling deadlocks. They are called:

1. Deadlock prevention
2. Deadlock avoidance
3. Deadlock detection

Each of these techniques is complex and we will discuss them in detail in our next month's column. Can you come up with a technique for each of these strategies on your own?

This month's takeaway problem

Consider the following code snippet where the main thread creates a child thread. The child thread increments the global counter and the main thread prints the value of the counter. The global counter is protected by a mutex lock. Can you point out whether this code snippet can deadlock?

```
void *counter_func(void *);
```

```
int count_var;
mutex_t count_var_lock;
```

```
main()
{
```

```
    char str[80];
    pthread_t child_thread_id;

    pthread_create(&child_thread_id, NULL, counter_func);

    while(1)
    {

        //read the next string
        Scanf ("%s", str);
        pthread_suspend(child_thread_id);

        mutex_lock(&count_var_lock);
        printf("count value is = %d\n", count_var);
        mutex_unlock(&count_var_lock);

        pthread_continue(child_thread_id);
    }

    return(0);
}

void *counter_func(void *arg)
{
    int i;

    while (1)
    {
        Printf("incrementing the counter value\n");
        mutex_lock(&count_var_lock);
        count_var++;

        //do nothing
        for (int i=0; i<MAXCNT; i++);

        mutex_unlock(&count_var_lock);

        //do nothing
        for (int j=0; j<MAXCNT; j++);
    }

    return((void *)0);
}
```

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, at sandyaasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

About the author:

Sandya Mannarswamy. The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools.